



Mini-informe Control de Congestion

Autor: Sebastian Cadena

Profesor: Ivana Bachmann

Repositorio: https://github.com/JuniorC-code/redes/tree/main/actividad_4-Congestion

En esta actividad se creara el código necesario para manejar Go Back-N a partir del código creado en la actividad anterior stop & wait.

Clase CongestionControl:

```python
class CongestionControl:
    def __init__(self, MSS: int):
        self.current_state = "slow start"
        self.MSS = MSS
        self.cwnd: int = MSS                    # Representa el total de bytes que puede tener la ventana de congestion
        self.ssthresh = None


    # Debe retornar el valor cwnd almacenado en congestionControl, corresponde al tamaño en bytes
    # Como cwnd representa un tamaño en bytes, su valor sigue siendo un entero (int), no un objeto bytes
    def get_cwnd(self) -> bytes:
        return self.cwnd

    # Añada a su clase CongestionControl la función get_MSS_in_cwnd()-> int. Esta función retorna la el tamaño de la ventana expresado como la cantidad de
    # MSSs completos que caben en cwnd. Por ejemplo si su función get_cwnd() retorna 17 bytes y MSS = 4, entonces get_MSS_in_cwnd() deberá retornar 4.

    def get_MSS_in_cwnd(self) -> int:
        return self.cwnd // self.MSS


    # Añada a su clase CongestionControl la función event_ack_received(). Esta función se encargará de manejar los cambios asociados a la recepción de ACKs.
    # Por ejemplo, si su current_state es slow start entonces recibir un ACK resultará en el aumento de cwnd en 1 MSS. Similarmente event_ack_received deberá
    # manejar el caso en que current_state sea igual congestion avoidance. Para los aumentos en congestion avoidance, note que cada aumento corresponde a una
    # fracción (1/self.get_MSS_in_cwnd()) de MSS. No olvide que al aumentar el cwnd
    # debe checkear si dicho aumento genera un cambio en el estado current_state. Si se genera un cambio de estado recuerde actualizar sus otras variables según sea necesario.

    def event_ack_received(self):
        if self.current_state == "slow start":
            self.cwnd += self.MSS

            if self.ssthresh is not None and self.cwnd >= self.ssthresh:
                self.current_state = "congestion avoidance"

        elif self.current_state == "congestion avoidance":
            mss_in_cwnd = self.get_MSS_in_cwnd()
            self.cwnd += self.MSS / mss_in_cwnd


    # Añada a su clase CongestionControl la función event_timeout(). Esta función se encargará de manejar los cambios asociados a que ocurra timeout.
    # Por ejemplo, si es la primera vez que ocurre timeout dentro de slow start entonces
    # deberá inicializar el valor de ssthresh. Esta función también deberá manejar los cambios de estado, de tamaño de ventana de congestión y de Slow Start threshold.

    # notar que como en ambos casos de cong avoidance como slow start se genera lo mismo, no es necesario usar if's para los diferentes casos...
    def event_timeout(self):
        self.ssthresh = self.cwnd // 2
        self.cwnd = self.MSS
        self.current_state = "slow start"

    def is_state_slow_start(self):
        return self.current_state == "slow start"

    def is_state_congestion_avoidance(self):
        return self.current_state == "congestion avoidance"

    def get_ssthresh(self):
        return self.ssthresh
```

Se inicializa la clase con solo cuatro atributos, el estado en el que se encuentra el control de congestión, el MSS (Maximum Segment Size), CWND (Congestion Window) y SSTRESH (Slow Start Threshold). Todos estos atributos son necesarios para poder definir como trabajaremos con los segmentos de la ventana en cada momento.

El resto de métodos son directos de la teoría en el control de congestión en redes.

Preguntas previas a el código de Go-back N:

- ¿Por qué le puede servir usar como base para *Go Back-N* el código de *Stop & Wait* de la sección anterior? ¿Qué similitud tiene *Go Back-N* con *Stop & Wait?*

- *Analizando a profundidad, es posible notar que Stop and wait es un caso particular de Go Back N, donde la ventana del emisor es de tamaño 1 MSS*
- *Tanto en Stop & Wait como en Go Back-N, el receptor descarta paquetes que no estén en orden o ACK's repetidos, solo acepta el segmento que tenga exactamente el número de secuencia esperado (expected_seq). Si llega un segmento adelantado o desordenado, lo descarta y reenvía el ACK del último paquete correcto recibido. Una de las mayores diferencias es que al cumplirse el timeout en stop&wait, se reenvía un único segmento, mientras que en Go-back N es el conjunto de segmentos de la ventana*.

- En la sección anterior solo se muestra la función send de *Stop & Wait*, ¿Qué ocurre con la función recv? ¿Esta función cambia con el uso de SocketUDP y SlidingWindowsCC?
- No debiera de diferir mucho la lógica entre ambos, pues go-back N no requiere que haya almacenamiento de los paquetes en el receptor... Esta función debería de solamente inspeccionar el segmeto que le llega y comprobar su correctitud, guardar datos según corresponda, avanzar su numero de seq. Esperado y devolver el ACK correspondiente. En caso que no, descarta segmento y reenvía el ultimo ACK valido.
- El cambio de la función recv no es muy grande. La idea principal es que con laa clase SlidingWindowCC podemos llevar el control de la ventana, la secuencia y el estado de congestión en el emisor. Y así, nuestro receptor solo debe de preocuparse de procesar los bytes y enviar las confirmaciones que correspondan...

Preguntas de parte 3:

- ¿Cómo se compara *Go-back N* con control de congestión versus sin control de congestión? ¿Cuál toma menos tiempo en transmitir un archivo de más de 1000 B?

- SIN control de congestión**:** Mantiene el mismo tamaño de ventana. Si se pierden paquetes o existen timeouts, se tendrá que reenviar ráfagas completas de segmentosvarias veces. Esto genera un número muy alto de mensajes, saturando así la conexión.
- CON control de congestión: Ajusta la ventana ($cwnd$) dinámicamente según el estado de la red. Al detectar un timeout, desploma $cwnd$ a 1 MSS, reduciendo la velocidad de envío. Esto resulta en muchos menos mensajes totales intercambiados, priorizando la estabilidad del enlace.

- Go-back N sin control de congestión podría ser mas rápido que con control. Esto debido a que para dicha cantidad tan chica de B, la ventana constantemente pasaría reduciéndose a 1 MSS y al estado slow start. No se saturaría la red con mensajes inútiles pero por esta misma razón podría tardar un poco mas. Mientras que aquella sin control, reenviaría muchos mensajes duplicados y por lo tanto, inútiles y descartables para el receptor.
  En una simulación local sin latencia de red real: